TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CUỐI KỲ
CÔNG NGHỆ LẬP TRÌNH ĐA NỀN TẢNG CHO ỨNG DỤNG DI ĐỘNG
ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG DANH BẠ

Giảng viên hướng dẫn: ThS. Nguyễn Thanh Truyền
Lớp: 10_ĐH_CNPM2
MSSV: 1050080096
Sinh viên thực hiện: Võ Minh Hân
Niên khóa: 2021-2025

TP. Hồ Chí Minh, tháng 5 năm 2025

TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CUỐI KỲ
CÔNG NGHỆ LẬP TRÌNH ĐA NỀN TẢNG CHO ỨNG DỤNG DI ĐỘNG
ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG DANH BẠ

Giảng viên hướng dẫn: ThS. Nguyễn Thanh Truyền

Lớp: 10_ĐH_CNPM2

MSSV: 1050080096

Sinh viên thực hiện: Võ Minh Hân

Niên khóa: 2021-2025

TP. Hồ Chí Minh, tháng 5 năm 2025

LỜI CẢM ƠN

Trước tiên, em xin gửi lời biết ơn chân thành đến Ban giám hiệu, Thầy Nguyễn Thanh Truyền trong khoa Công nghệ thông tin , người đã tận tình hướng dẫn, giúp đỡ và truyền đạt kiến thức quý báu trong suốt quá trình học tập và hoàn thành đề tài này.

Em cũng xin cảm ơn các bạn bè đã luôn động viên, hỗ trợ tôi về tinh thần cũng như chia sẻ những kinh nghiệm quý giá trong suốt quá trình nghiên cứu và hoàn thiện công trình.

Cuối cùng, em xin gửi lời cảm ơn sâu sắc đến gia đình đã luôn là nguồn động lực vững chắc, tạo điều kiện tốt nhất để tôi có thể tập trung học tập và nghiên cứu.

Em chân thành cảm ơn tất cả mọi người đã đồng hành cùng tôi trong suốt quá trình này!

Mục Lục

CHƯƠNG I: GIỚI THIỆU CHUNG 6
1.1 Tổng quan đề tài (what) 6
1.2 Nền tảng triển khai (where) 6
1.3 Thời gian thực hiện (when) 6
1.4 Lý do chọn đề tài (why) 6
1.5 Công nghệ và phương pháp thực hiện (how) 6
CHƯƠNG II: CẤU TRÚC VÀ CHỨC NĂNG ỨNG DỤNG 7
2.1 Kiểm tra chức năng lấy danh sách liên hệ sử dụng Firebase (Firebase). 7
2.2 Màn hình đăng ký 8
2.2.1 Mô tả (What) 8
2.2.2 Vị trí hiển thị (Where) 8
2.2.3 Thời điểm kích hoạt (When) 8
2.2.4 Mục đích (Why) 8
2.2.5 Triển khai (How) 8
2.2.6 Giao diện 11
2.3 Màn hình đăng nhập 12
2.3.1 Mô tả (What) 12
2.3.2 Vị trí hiển thị (Where) 12
2.3.3 Thời điểm kích hoạt (When) 12
2.3.4 Mục đích (Why) 12
2.3.5 Triển khai (How) 13
2.3.6 Giao diện 15
2.4 Màn hình chính home 16
2.4.1 Mô tả (What) 16
2.4.2 Vị trí hiển thị (Where) 16
2.4.3 Thời điểm kích hoạt (When) 16
2.4.4 Mục đích (Why) 16
2.4.5 Triển khai (How) 17

2.4.6 Giao diện 20
2.5 Màn hình admin 21
2.5.1 Mô tả (What) 21
2.5.2 Vị trí hiển thị (Where) 21
2.5.3 Thời điểm kích hoạt (When) 21
2.5.4 Mục đích (Why) 21
2.5.5 Triển khai (How) 21
2.5.6 Giao diện 25
TÀI LIỆU THAM KHẢO 26

CHƯƠNG I: GIỚI THIỆU CHUNG

1.1 Tổng quan đề tài (what)

Đề tài tập trung vào việc xây dựng một ứng dụng quản lý danh bạ cá nhân dành cho người dùng thông thường và quản trị viên (admin), cho phép đăng nhập phân quyền, thêm/sửa/xóa liên hệ, tìm kiếm và gọi điện. Ứng dụng hỗ trợ đăng ký, đăng nhập bằng email/password và Google, sử dụng dịch vụ Firebase để lưu trữ và đồng bộ dữ liệu thời gian thực.

1.2 Nền tảng triển khai (where)

Ứng dụng được phát triển trên nền tảng Flutter – một framework mã nguồn mở của Google dùng để xây dựng ứng dụng di động. Hệ thống backend được triển khai trên Firebase (Authentication và Firestore). Thiết bị đích là điện thoại Android.

1.3 Thời gian thực hiện (when)

Thời gian thực hiện đề tài 4 tuần . Đây là giai đoạn thực hiện báo cáo cuối kỳ trong chương trình đào tạo.

1.4 Lý do chọn đề tài (why)

Quản lý danh bạ là nhu cầu phổ biến với mọi người dùng điện thoại. Tuy nhiên, các ứng dụng mặc định thiếu tính năng phân quyền, đồng bộ và mở rộng. Đề tài giúp người học hiểu rõ hơn về quy trình phát triển ứng dụng di động tích hợp dịch vụ đám mây, đồng thời rèn luyện kỹ năng lập trình thực tiễn và giải quyết bài toán bảo mật, phân quyền.

1.5 Công nghệ và phương pháp thực hiện (how)

- Ngôn ngữ/lập trình: Dart (Flutter) .
- Cơ sở dữ liệu: Firebase Firestore để lưu trữ thông tin liên hệ.
- Xác thực: Firebase Authentication (email/password + Google).
- Tính năng gọi điện: Dùng package url_launcher để thực hiện cuộc gọi trực tiếp.

CHƯƠNG II: CẤU TRÚC VÀ CHỨC NĂNG ỨNG DỤNG

2.1 Kiểm tra chức năng lấy danh sách liên hệ sử dụng Firebase (Firebase).

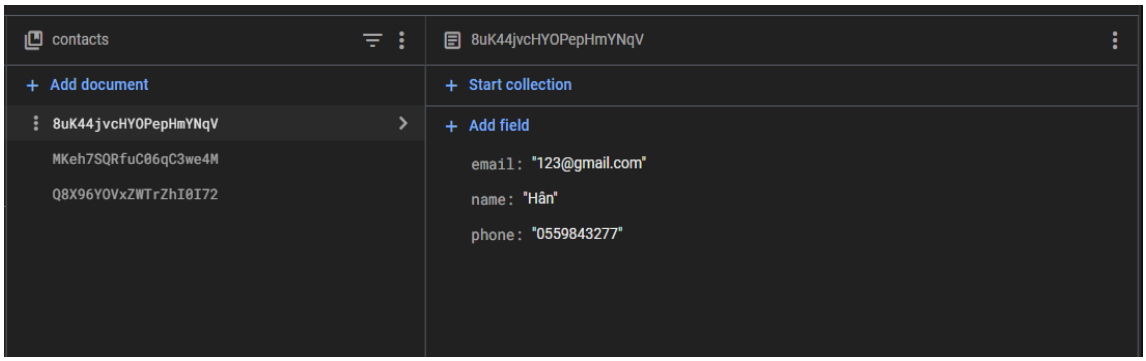
Trong ứng dụng danh bạ, chức năng **lấy danh sách liên hệ** đóng vai trò quan trọng, cho phép người dùng xem toàn bộ các liên hệ đã được lưu trữ. Mặc dù chức năng này **chưa được thực hiện trong mã nguồn của bài làm**, nhưng về mặt lý thuyết, có thể mô tả như sau:.

Mục tiêu

- Kiểm tra khả năng truy xuất dữ liệu liên hệ từ cơ sở dữ liệu Firebase.
- Mô tả quy trình hoạt động lý thuyết của chức năng lấy danh sách liên hệ trong ứng dụng.

Mô hình hoạt động

- Cơ sở dữ liệu sử dụng: Firebase (Realtime Database hoặc Cloud Firestore).
- Cấu trúc dữ liệu mẫu :



Quy trình hoạt động:

- Ứng dụng gửi yêu cầu (request) tới Firebase để truy xuất dữ liệu tại node contacts.
- Firebase trả về danh sách liên hệ dưới dạng dữ liệu JSON.
- Ứng dụng xử lý dữ liệu và hiển thị danh sách liên hệ trên giao diện người dùng

Chức năng **lấy danh sách liên hệ từ Firebase** là một trong những tính năng cốt lõi trong các ứng dụng danh bạ hiện đại. Mặc dù chưa triển khai trong bài làm, nhưng chức năng này hoàn toàn có thể được tích hợp trong tương

lai bằng cách sử dụng các thư viện như firebase_core, cloud_firestore hoặc firebase_database trong Flutter.

2.2 Màn hình đăng ký

2.2.1 Mô tả (What)

Chức năng đăng ký tài khoản là email từ Firebase. Người dùng nhập email và mật khẩu để tạo tài khoản mới.

2.2.2 Vị trí hiển thị (Where)

Khi vào ứng dụng , màn hình Login sẽ hiện lên. Người dùng sẽ thấy nút "Sing up" nằm ở phía dưới hoặc gần phần Login.

2.2.3 Thời điểm kích hoạt (When)

Sau khi người dùng vào hệ thống màn hình Login sẽ hệ lên, người dùng chọn Singup để tạo tài khoản, màn hình sẽ chuyển sang Singup .

2.2.4 Mục đích (Why)

- Cho phép người dùng tạo ra tài khoản cá nhân để đăng nhập vào hệ thống
- Tăng tính cá nhân hóa, bảo mật và khả năng lưu trữ thông tin người dùng trên nền tảng Firebase.

2.2.5 Triển khai (How)

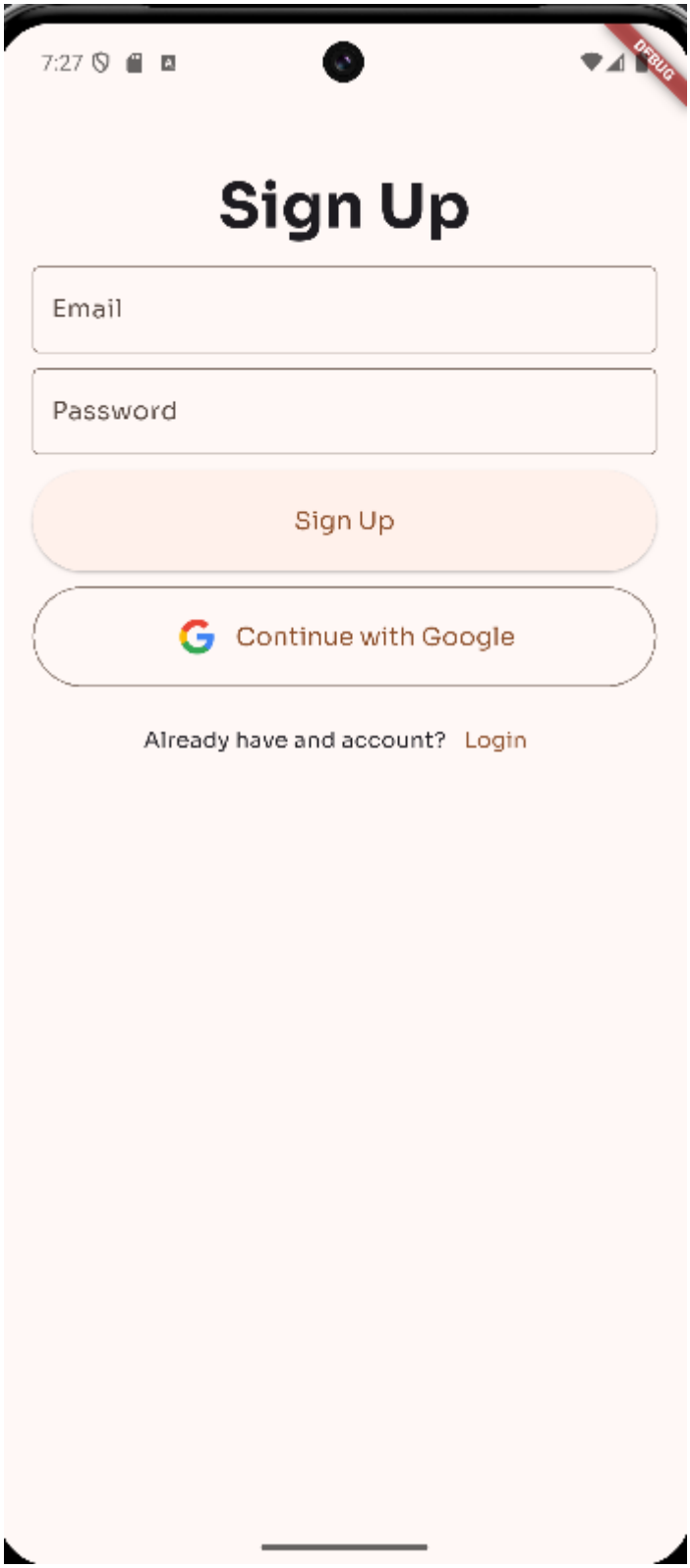
- Sử dụng Firebase Authentication để xử lý chức năng đăng ký tài khoản mới bằng email và mật khẩu.
- Giao diện đăng ký được xây dựng bằng TextFormField cho hai trường: Email và Password. Việc kiểm tra dữ liệu được thực hiện bằng thuộc tính validator trong Form.
- Khi người dùng nhấn nút "Sign Up", ứng dụng kiểm tra tính hợp lệ của dữ liệu đầu vào, sau đó gọi phương thức createUserWithEmailAndPassword() thông qua lớp AuthService.
- Nếu tạo tài khoản thành công, người dùng sẽ được điều hướng đến màn hình chính thông qua Navigator.pushReplacementNamed.

- Nếu có lỗi (ví dụ như trùng email, định dạng không hợp lệ...), thông báo lỗi sẽ được hiển thị bằng SnackBar.
- Ngoài ra, nút "Continue with Google" sử dụng đăng nhập Google thông qua GoogleSignIn và Firebase. Khi đăng nhập Google thành công, người dùng cũng được điều hướng đến màn hình chính.

```
26          Text(
27            "Sign Up",
28            style:
29                GoogleFonts.sora(fontSize: 40, fontWeight: FontWeight.w700),
30          ),  // Text
31          const SizedBox(
32            height: 10,
33          ),  // SizedBox
34          SizedBox(
35              width: MediaQuery.of(context).size.width * .9,
36              child: TextFormField(
37                validator: (value) =>
38                    value!.isEmpty ? "Email cannot be empty." : null,
39                controller: _emailController,
40                decoration: const InputDecoration(
41                  border: OutlineInputBorder(),
42                  label: Text("Email"),
43                ),  // InputDecoration
44              )),  // TextFormField, SizedBox
45          const SizedBox(
46            height: 10,
47          ),  // SizedBox
48          SizedBox(
49              width: MediaQuery.of(context).size.width * .9,
50              child: TextFormField(
51                validator: (value) => value!.length < 8
52                    ? "Password should have atleast 8 characters."
53                    : null,
54                controller: _passwordController,
55                obscureText: true,
56                decoration: const InputDecoration(
57                  border: OutlineInputBorder(),
58                  label: Text("Password"),
59                ),  // InputDecoration
60              )),  // TextFormField, SizedBox
61          const SizedBox(
62            height: 10,
63          ),  // SizedBox
64          SizedBox(
65              height: 65,
```

```
const SizedBox(
  height: 10,
), // SizedBox
SizedBox(
    height: 65,
    width: MediaQuery.of(context).size.width * .9,
    child: ElevatedButton(
        onPressed: () {
          if (formKey.currentState!.validate()) {
            AuthService()
                .createAccountWithEmail(
                    _emailController.text, _passwordController.text)
                .then((value) {
              if (value == "Account Created") {
                ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(content: Text("Account Created")));
                Navigator.pushReplacementNamed(context, "/home");
              } else {
                ScaffoldMessenger.of(context).showSnackBar(SnackBar(
                  content: Text(
                    value,
                    style: TextStyle(color: Colors.white),
                  ), // Text
                  backgroundColor: Colors.red.shade400,
                ));  // SnackBar
              }
            });
          }
        },
        child: const Text(
          "Sign Up",
          style: TextStyle(fontSize: 16),
```

2.2.6 Giao diện



2.3 Màn hình đăng nhập

2.3.1 Mô tả (What)

Màn hình đăng nhập là nơi người dùng nhập **email** và **mật khẩu** để truy cập vào hệ thống. Đây là một phần của chức năng xác thực người dùng thông qua Firebase Authentication.

2.3.2 Vị trí hiển thị (Where)

Màn hình đăng nhập hiển thị **ngay sau khi mở ứng dụng**, là giao diện đầu tiên mà người dùng nhìn thấy khi đăng nhập.

Tai đây người dùng sẽ thấy

- Các trường nhập email và mật khẩu.
- Nút "Login" để đăng nhập.
- Liên kết hoặc nút "Sign Up" để chuyển sang màn hình đăng ký tài khoản (nếu chưa có tài khoản).

2.3.3 Thời điểm kích hoạt (When)

Màn hình đăng nhập được kích hoạt khi:

- Người dùng mở ứng dụng lần đầu.
- Người dùng đăng xuất khỏi ứng dụng.
- Phiên đăng nhập hết hạn.

2.3.4 Mục đích (Why)

- Xác thực danh tính người dùng.
- Đảm bảo rằng chỉ người đã đăng ký mới có thể truy cập các chức năng chính của ứng dụng.
- Bảo mật thông tin cá nhân và danh bạ của người dùng.

2.3.5 Triển khai (How)

- Người dùng nhập email và mật khẩu vào hai ô TextFormField.
- Sau khi kiểm tra hợp lệ (form validate), gọi AuthService().loginWithEmail() để đăng nhập.
- Nếu đăng nhập thành công:

  Gọi getCurrentUserData() để kiểm tra role từ Firestore:

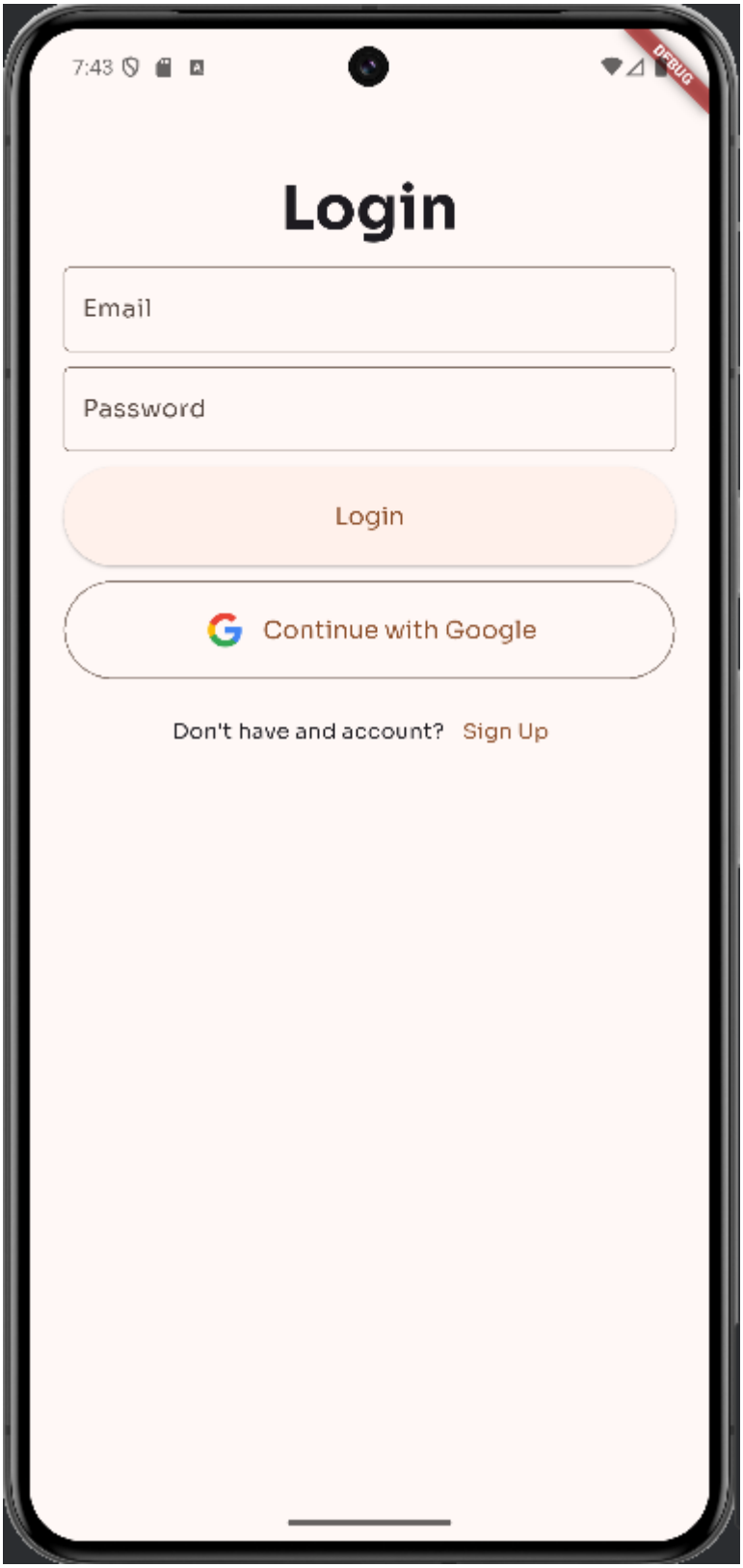
  Nếu là admin → chuyển đến AdminDashboard()

  Nếu là user → chuyển đến Homepage(), đồng thời hiện SnackBar thông báo
- Nếu đăng nhập không thành công → Hiển thị SnackBar báo lỗi.
- Ngoài ra, người dùng có thể chọn **Continue with Google** để sử dụng tài khoản Google thông qua continueWithGoogle().

```
SizedBox(
    width: MediaQuery.of(context).size.width * .9,
    child: TextFormField(
      validator: (value) =>
          value!.isEmpty ? "Email cannot be empty." : null,
      controller: _emailController,
      decoration: const InputDecoration(
        border: OutlineInputBorder(),
        label: Text("Email"),
      ),  // InputDecoration
    )),  // TextFormField, SizedBox
const SizedBox(
  height: 10,
),  // SizedBox
SizedBox(
    width: MediaQuery.of(context).size.width * .9,
    child: TextFormField(
      validator: (value) => value!.length < 8
          ? "Password should have atleast 8 characters."
          : null,
      controller: _passwordController,
      obscureText: true,
      decoration: const InputDecoration(
        border: OutlineInputBorder(),
        label: Text("Password"),
      ),  // InputDecoration
    )),  // TextFormField, SizedBox
const SizedBox(
  height: 10,
),  // SizedBox
```

```
SizedBox(
    height: 65,
    width: MediaQuery.of(context).size.width * .9,
    child: ElevatedButton(
        onPressed: () {
          if (formKey.currentState!.validate()) {
            AuthService()
                .loginWithEmail(
                    _emailController.text, _passwordController.text)
                .then((value) async {
              if (value == "Login Successful") {


                bool admin = await getCurrentUserData();
                if(admin == true){
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(builder: (context) => AdminDashboard())
                  );
                }else {
                  ScaffoldMessenger.of(context).showSnackBar(
                      const SnackBar(content: Text("Login user")));
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(builder: (context) => Homepage()), // T
                  );
```

2.3.6 Giao diện



2.4 Màn hình chính home

2.4.1 Mô tả (What)

Màn hình chính của ứng dụng là nơi hiển thị **danh sách các liên hệ** mà người dùng đã lưu. Mỗi liên hệ bao gồm:

- Tên người liên hệ.
- Số điện thoại.
- Biểu tượng gọi điện để thực hiện cuộc gọi trực tiếp.

  Ngoài ra, còn có:
- Ô tìm kiếm để lọc liên hệ theo tên hoặc số điện thoại.
- Nút thêm liên hệ (icon dấu "+" ở góc dưới bên phải).

2.4.2 Vị trí hiển thị (Where)

- Hiển thị ngay sau khi người dùng đăng nhập thành công.
- Là giao diện chính mà người dùng tương tác để xem, tìm kiếm, gọi điện hoặc thêm mới liên hệ.

2.4.3 Thời điểm kích hoạt (When)

- Sau khi người dùng đăng nhập vào ứng dụng.
- Khi người dùng trở về từ màn hình thêm, sửa hoặc chi tiết liên hệ.

2.4.4 Mục đích (Why)

- Cho phép người dùng dễ dàng quản lý danh bạ cá nhân.
- Thực hiện thao tác nhanh chóng như: tìm kiếm, gọi điện, thêm liên hệ mới.
- Cung cấp giao diện chính để điều hướng đến các chức năng khác của ứng dụng.

2.4.5 Triển khai (How)

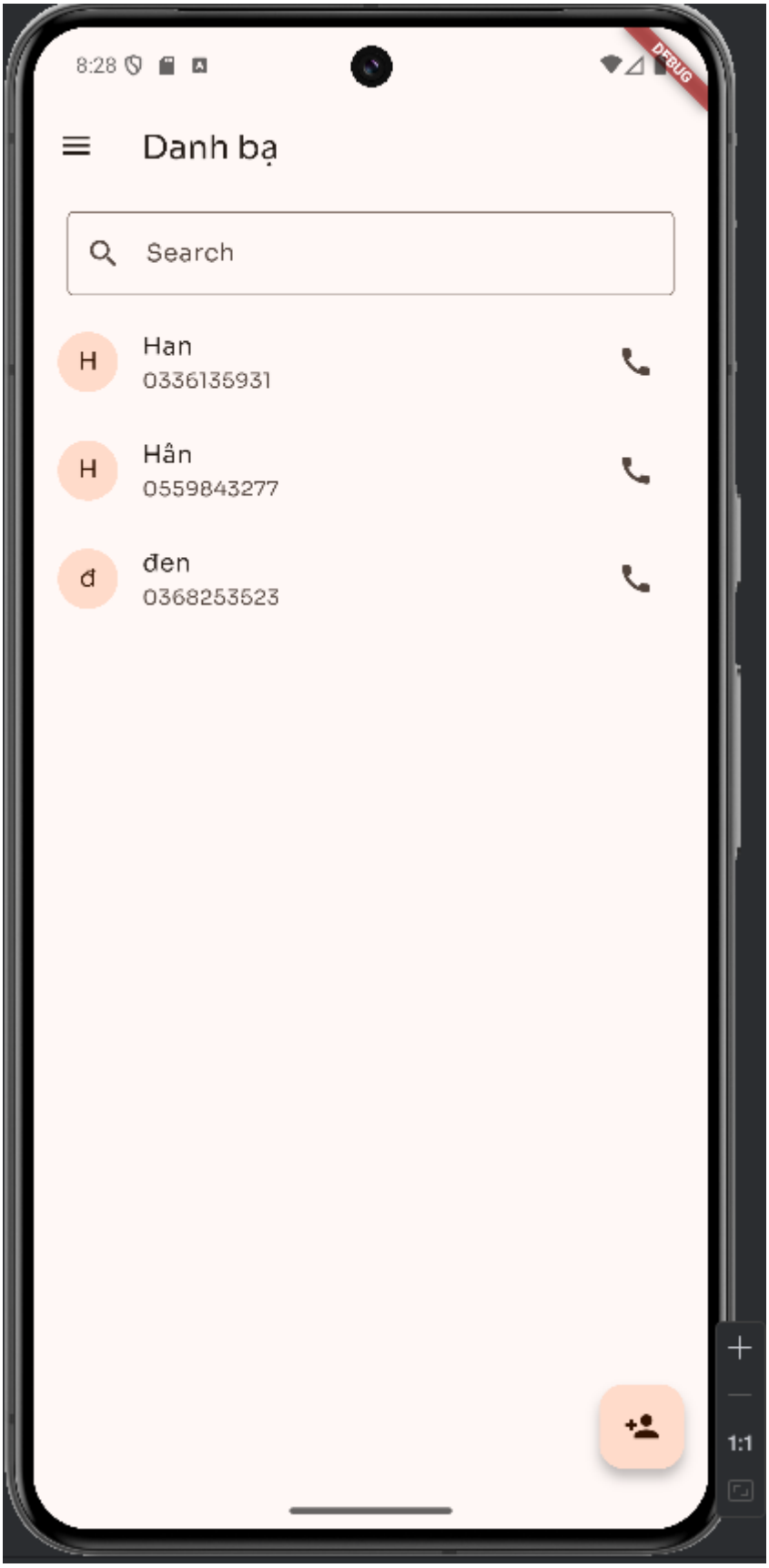
- Sử dụng Firebase Realtime Database hoặc Firestore để lưu trữ danh sách liên hệ.
- Dữ liệu được đọc bằng cách sử dụng StreamBuilder hoặc FutureBuilder trong Flutter để lắng nghe và hiển thị dữ liệu theo thời gian thực.
- Giao diện danh sách sử dụng ListView.builder để hiển thị từng liên hệ.
- Nút gọi điện được tích hợp sử dụng package url_launcher.
- Nút thêm mới dẫn đến màn hình tạo liên hệ (AddContactScreen).

```
Future<void> callUser(String phoneNumber) async {
  final Uri launchUri = Uri(scheme: 'tel', path: phoneNumber);

  try {
    if (await canLaunchUrl(launchUri)) {
      await launchUrl(launchUri);
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Không thể gọi số: $phoneNumber')),
      );
    }
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Lỗi khi gọi số: $phoneNumber')),
    );
  }
}
```

```
        title: Text("Danh bạ"),
        // search box
        bottom: PreferredSize(
            preferredSize: Size(MediaQuery.of(context).size.width * 8, 80),
            child: Container(
              padding: EdgeInsets.symmetric(vertical: 8),
              child: SizedBox(
                  width: MediaQuery.of(context).size.width * .9,
                  child: TextFormField(
                    onChanged: (value) {
                      searchContacts(value);
                      setState(() {});
                    },
                    focusNode: _searchfocusNode,
                    controller: _searchController,
                    decoration: InputDecoration(
                        border: OutlineInputBorder(),
                        label: Text("Search"),
                        prefixIcon: Icon(Icons.search),
                        suffixIcon: _searchController.text.isNotEmpty
                            ? IconButton(
                                onPressed: () {
                                  _searchController.clear();
                                  _searchfocusNode.unfocus();
                                  _stream = CRUDService().getContacts();
                                  setState(() {});
                                },
                                icon: Icon(Icons.close),
                              )  // IconButton
                            : null),  // InputDecoration
                  )),  // TextFormField, SizedBox
            )),  // Container, PreferredSize
      ),  // AppBar
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          Navigator.pushNamed(context, "/add");
        },
        child: Icon(Icons.person_add),
      ),  // FloatingActionButton
      drawer: Drawer(
```

```
} else {
  return ListView(
        children: snapshot.data!.docs
            .map((DocumentSnapshot document) {
              Map<String, dynamic> data =
                  document.data()! as Map<String, dynamic>;
              return ListTile(
                onTap: () => Navigator.push(
                    context,
                    MaterialPageRoute(
                        builder: (context) => UpdateContact(
                            name: data["name"],
                            phone: data["phone"],
                            email: data["email"],
                            docID: document.id))),  // UpdateContact, MaterialPa
                leading: CircleAvatar(child: Text(data["name"][0])),
                title: Text(data["name"]),
                subtitle: Text(data["phone"]),
                trailing: IconButton(
                  icon: Icon(Icons.call),
                  onPressed: () {
                    callUser(data["phone"]);
                  },
                ),  // IconButton
              );  // ListTile
            })
            .toList()
            .cast(),
      );  // ListView
}
),  // StreamBuilder
Scaffold
```

2.4.6 Giao diện



2.5 Màn hình admin

2.5.1 Mô tả (What)

Màn hình quản trị (Admin Dashboard) dành riêng cho người dùng có vai trò admin. Chức năng chính bao gồm:

- Hiển thị danh sách toàn bộ người dùng đã đăng ký.
- Hiển thị vai trò người dùng: user hoặc admin.
- Cho phép admin xóa người dùng khỏi hệ thống (nút thùng rác).
- Biểu đồ thống kê số lượng liên hệ mỗi người dùng đã tạo.

2.5.2 Vị trí hiển thị (Where)

- Được truy cập sau khi đăng nhập bằng tài khoản có quyền admin.
- Có thể từ màn hình Home chọn "Login" để chuyển đến trang đăng nhập dành cho admin, sau đó sẽ đến màn hình này.

2.5.3 Thời điểm kích hoạt (When)

Sau khi xác thực tài khoản là admin, hệ thống sẽ điều hướng đến màn hình Admin Dashboard..

2.5.4 Mục đích (Why)

- Quản lý người dùng: xem danh sách, phân quyền, xóa người dùng.
- Theo dõi hoạt động người dùng: biểu đồ thống kê số lượng liên hệ giúp admin đánh giá mức độ sử dụng.

2.5.5 Triển khai (How)

- Dữ liệu người dùng được lưu trữ trong Firebase Authentication và/hoặc Firestore.
- Danh sách người dùng được lấy từ Firestore, mỗi bản ghi gồm: email, role, số lượng liên hệ.
- Giao diện hiển thị sử dụng ListView.builder.
- Chức năng xóa người dùng: nút thùng rác gọi hàm xóa tài khoản khỏi cơ sở dữ liệu.
- Biểu đồ sử dụng thư viện như fl_chart hoặc charts_flutter để hiển thị dữ liệu thống kê dạng cột.

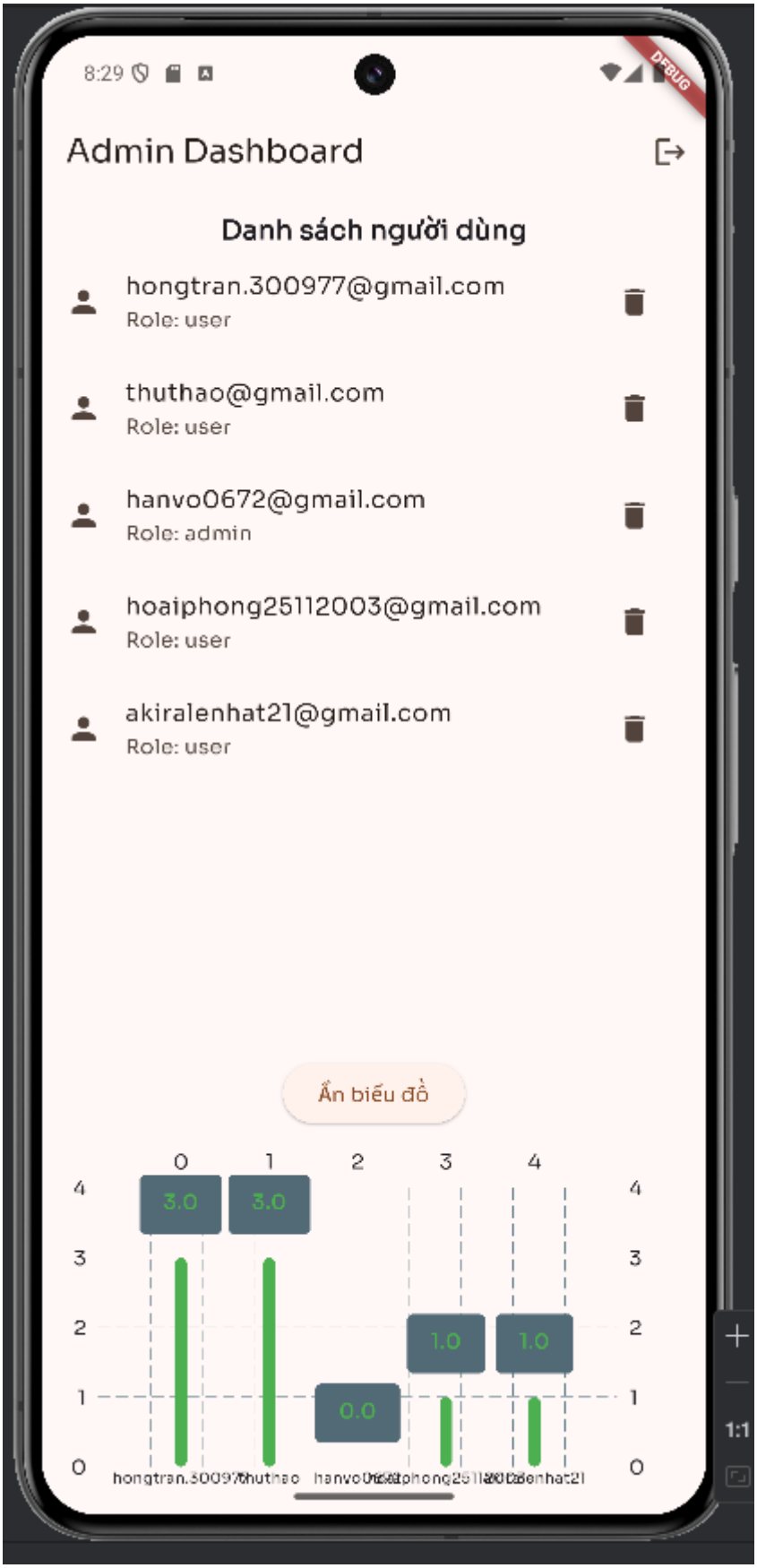
- Nút ẩn/hiện biểu đồ giúp cải thiện trải nghiệm người dùng.

```
  body: Column(
    children: [
      const SizedBox(height: 10),
      const Text(
        "Danh sách người dùng",
        style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
      ),  // Text
      Expanded(child: _buildUserList()),
      const SizedBox(height: 20),
      ElevatedButton(
        onPressed: () {
          setState(() {
            _showChart = !_showChart;
          });
        },
        child: Text(_showChart ? "Ẩn biểu đồ" : "Hiển thị biểu đồ danh bạ"),
      ),  // ElevatedButton
      const SizedBox(height: 10),
      if (_showChart) SizedBox(height: 250, child: _buildContactChart()),
    ],
  ),  // Column
);  // Scaffold
```

```
Widget _buildUserList() {
  return StreamBuilder<QuerySnapshot>(
    stream: FirebaseFirestore.instance.collection("users").snapshots(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: CircularProgressIndicator());
      } else if (snapshot.hasData) {
        var users = snapshot.data!.docs;
        return ListView.builder(
          itemCount: users.length,
          itemBuilder: (context, index) {
            var user = users[index];
            final data = user.data() as Map<String, dynamic>?;

            if (data == null) return const SizedBox();

            return ListTile(
              leading: const Icon(Icons.person),
              title: Text(data['email'] ?? 'Không có email'),
              subtitle: Text("Role: ${data['role'] ?? 'unknown'}"),
              trailing: IconButton(
                icon: const Icon(Icons.delete),
                onPressed: () {
                  FirebaseFirestore.instance
                      .collection("users")
                      .doc(user.id)
                      .delete();
                },
              ),  // IconButton
            );  // ListTile
          },
        );  // ListView.builder
      } else if (snapshot.hasError) {
        return Center(child: Text("Error: ${snapshot.error}"));
      } else {
        return const SizedBox();
      }
    },
  );  // StreamBuilder
}
```

```
Widget _buildContactChart() {
  return FutureBuilder<QuerySnapshot>(
    future: FirebaseFirestore.instance.collection("users").get(),
    builder: (context, snapshot) {
      if (!snapshot.hasData) return const Center(child: CircularProgressIndicator());

      var docs = snapshot.data!.docs;

      return FutureBuilder<List<int>>(
        future: Future.wait(docs.map((doc) async {
          var contactsSnap = await doc.reference.collection("contacts").get();
          return contactsSnap.size; // Số lượng contacts
        }).toList()),
        builder: (context, countSnapshot) {
          if (!countSnapshot.hasData) return const Center(child: CircularProgressIndicator());

          List<int> contactCounts = countSnapshot.data!;
          double maxY = 0;
          List<BarChartGroupData> barGroups = [];
          List<String> emails = [];

          for (int i = 0; i < docs.length; i++) {
            final data = docs[i].data() as Map<String, dynamic>;
            final email = data['email'] ?? 'unknown';
            final count = contactCounts[i];

            emails.add(email);
            if (count > maxY) maxY = count.toDouble();

            barGroups.add(
              BarChartGroupData(
                x: i,
                barRods: [
                  BarChartRodData(toY: count.toDouble(), color: Colors.green),
                ],
                showingTooltipIndicators: [0],
              ),  // BarChartGroupData
            );
```

2.5.6 Giao diện



TÀI LIỆU THAM KHẢO

[1] F. f. B. –. A. Biessek, 2019.

[2] Programming Flutter: Native, Cross-Platform Apps the Easy Way – Carmine Zaccagnino, 2020.

[3] Firebase Essentials – Android Edition – Neil Smyth, 2020.

[4] Learning Dart – Ivo Balbaert & Dzenan Ridjanovic, 2015.